



YARIŞMA
TEKNOFEST

KATEGORİ
Döner Kanat IHA

HAZIRLAYAN
İSMAİL FATİH
ÇOLAK

Yazılım Tasarımı

2025 / 2026 DÖNEMİ



DOKÜMAN TİPİ

YAZILIM TASARIM
DOKÜMANI

TAKIM

SAÜRO IHA

DÖNEM

2025/2026

İÇİNDEKİLER

1

GİRİŞ

Dokümanın Amacı ve Kapsamı.....	3
Proje Tanıtımı.....	5
Tanımlar ve Kısaltmalar.....	6
Görev Tanımı	8
Yazılımdan Beklenenler.....	9

2

İHTİYAÇLAR

Yazılımsal İhtiyaçlar	11
Donanımsal İhtiyaçlar	14

3

TASARIM

Genel Yazılım Mimarisi	15
Yazılım Bileşenleri ve Görevleri	17
Durum Makinesi	25
Veri Akışı ve İletişim	28
Güvenlik ve Fail-Safe Tasarım	35
Konfigürasyon & Parametreler	38
Test ve Doğrulama	41

4

SONUÇ

Tasarım Kararlarının Uyumu	44
Tasarımın Hedeflerle Gerekçeleri	45
Kısıtlar ve Bilinçli Sınırlar	46
Sonraki Aşamalar	46

01 Giriş

Bu bölüm, 2025–2026 TEKNOFEST Döner Kanat İHA yarışması kapsamında geliştirilen SAÜRO İHA yazılımına ait tasarım dokümanının genel çerçevesini sunmaktadır.

Doküman; analiz aşamasında belirlenen gereksinimler doğrultusunda şekillendirilen yazılım mimarisini, tasarım yaklaşımlarını ve sistem bileşenlerini açıklamayı amaçlamaktadır.

İçerik, uygulama ve kod detaylarına girilmeden, yazılımın nasıl bir yapı ile tasarlandığını ortaya koyan yüksek seviyeli bir referans belge niteliğindedir.



MODÜLER

Yazılım, bakım ve genişletilebilirliği artırmak amacıyla modüler bir mimari ile tasarlanmıştır.



İZLENEBİLİR

Görev yürütme süreci, insan müdahalesine ihtiyaç duymadan çalışan durum makinesi tabanlı bir yapı ile yönetilmektedir.



GÜVENLİ

Yazılım, olası hata durumlarında kontrollü ve güvenli davranış sergileyecek fail-safe mekanizmalar içermektedir.

02 Amaç

Bu dokümanın amacı, SAÜRO İHA sistemine ait yazılım tasarımını açık, tutarlı ve sistematik bir şekilde ortaya koymaktır.

Bu kapsamda; yazılımın genel mimarisi, temel bileşenleri, durum bazlı kontrol yapısı, veri akışı ve güvenlik odaklı tasarım yaklaşımları tanımlanmaktadır.

Hazırlanan tasarımın temel hedefi, yarışma görevlerini insan müdahalesine minimum seviyede ihtiyaç duyacak şekilde, güvenilir ve izlenebilir bir biçimde gerçekleştirebilecek bir yazılım altyapısı oluşturmaktır.

01 Kapsam

Bu bölümde, SAÜRO İHA yazılım tasarım dokümanının kapsadığı konular ve bilinçli olarak kapsam dışında bırakılan alanlar tanımlanmaktadır. Dokümanın kapsamı, yazılım tasarımının sınırlarını netleştirmek ve dokümanın hangi seviyede ele alındığını açıkça ortaya koymak amacıyla belirlenmiştir.



02 Kapsam içi ve dışı Konular

KAPSAM İÇİ	KAPSAM DIŞI
Görev yürütme ve State machine yapısı	Algoritmik ve matematiksel detaylar
Genel yazılım mimarisi	Görüntü işleme parametreleri
Güvenlik ve fail-safe tasarım yaklaşımları	PID ayarları ve katsayıları
Bileşenler arası veri akışı ve iletişim	Donanım montaj detayları
Yer İstasyonu ile iletişim	Yazılımın kod seviyesindeki uygulama detayları
Yazılım bileşenleri ve sorumlulukları	Tamamlanmış kod yapıları

03 Hedef Kitle

Doküman Aşağıdaki Kitleleri Hedefler:

- Yazılım Geliştirme Ekibi
- Sistem Entegrasyonundan Sorumlu Ekip Üyeleri
- Yarışma jüri ve değerlendiricileri

1.2 Proje Tanıtımı

SAÜRO İHA projesi, 2025–2026 TEKNOFEST Döner Kanat İHA yarışması kapsamında geliştirilen, otonom görev icra edebilen bir insansız hava aracı sistemini kapsamaktadır. Proje, yarışma senaryoları doğrultusunda belirlenen görevleri, minimum insan müdahalesi ile güvenli ve tekrarlanabilir şekilde gerçekleştirmeyi hedeflemektedir.

Sistem; uçuş kontrolü, görev yönetimi, görüntü işleme ve yer istasyonu etkileşimi gibi farklı alt bileşenlerden oluşan bütüncül bir mimari üzerine kuruludur. Bu mimaride, uçuş kontrolü görevleri uçuş bilgisayar (flight controller) tarafından yürütülürken; görev mantığı, algılama ve üst seviye karar verme süreçleri yardımcı bilgisayar (companion computer) üzerinde çalışan yazılım tarafından yönetilmektedir.

SAÜRO İHA yazılımı, yarışma süresince gerçekleştirilecek görevlerin gerektirdiği otonomluk seviyesini sağlayacak şekilde tasarlanmıştır. Yazılım; görevlerin durum bazlı bir yapı ile yönetilmesini, sensör ve kamera verilerinin etkin şekilde kullanılmasını ve sistemin anormal durumlar karşısında güvenli davranış sergilemesini mümkün kılmaktadır.

Proje kapsamında geliştirilen yazılım altyapısı, simülasyon ortamında test edilerek doğrulanmakta ve elde edilen kazanımların gerçek sisteme aktarılması hedeflenmektedir. Bu yaklaşım, sistemin geliştirme sürecinde karşılaşılabilecek risklerin azaltılmasını ve entegrasyon sürecinin daha kontrollü ilerlemesini sağlamaktadır.

Bu dokümanda ele alınan yazılım tasarımı, SAÜRO İHA sisteminin görev gereksinimlerini karşılayacak, genişletilebilir ve sürdürülebilir bir yazılım mimarisi oluşturmayı amaçlamaktadır.

TANIMLAR VE KISALTMALAR	
TERİM	AÇIKLAMA
İHA	İnsansız Hava Aracı. İnsan müdahalesi olmadan veya sınırlı operatör kontrolü ile uçuş gerçekleştirebilen hava aracı.
SAÜRO	Proje kapsamında geliştirilen Döner Kanat İHA sisteminin adı.
ROS 2	Robot Operating System 2. Dağıtık, mesaj tabanlı ve gerçek zamanlı sistemleri destekleyen robot yazılım çatısı.
Node	ROS 2 içerisinde çalışan, belirli bir sorumluluğu olan yazılım bileşeni.
Topic	ROS 2 node'ları arasında veri alışverişi sağlayan yayın-abonelik tabanlı iletişim kanalı.
Service	ROS 2 içerisinde senkron, istek-yanıt tabanlı iletişim mekanizması.
Action	ROS 2 içerisinde uzun süren ve geri bildirim gerektiren işlemler için kullanılan iletişim yapısı.
Mission Manager Node	Görev yürütme sürecini yöneten, durum makinesini çalıştıran merkezi yazılım bileşeni.
State Machine (Durum Makinesi)	Görev yürütme sürecini belirli durumlar ve bu durumlar arasındaki geçişler üzerinden yöneten kontrol yapısı.
FAILSAFE	Sistem güvenliğini tehdit eden bir durum oluştuğunda devreye giren, görevi sonlandırarak güvenli uçuş davranışını tetikleyen durum.
RTL (Return to Launch)	İHA'nın kalkış noktasına otomatik olarak geri dönmesini sağlayan uçuş modu.
LAND	İHA'nın güvenli şekilde iniş gerçekleştirmesini sağlayan uçuş modu.
Pixhawk	İHA'nın uçuş kontrolünden sorumlu olan uçuş bilgisayarı (flight controller).
Uçuş Bilgisayarı	İHA'nın stabilizasyonu, uçuş modları ve düşük seviye kontrolünden sorumlu donanım birimi.
Yardımcı Bilgisayar	Görev mantığı, algılama ve yüksek seviye yazılım bileşenlerini çalıştıran bilgisayar (Raspberry Pi).
Raspberry Pi (Pi)	Yardımcı bilgisayar olarak kullanılan, Linux tabanlı tek kart bilgisayar.
Yer İstasyonu (Yi)	Operatörün sistemi izlediği ve kontrol ettiği, görev komutlarını gönderdiği yazılım/hardware ortamı.

TANIMLAR VE KISALTMALAR	
TERİM	AÇIKLAMA
Raw Image	Sıkıştırılmamış, ham kamera görüntü verisi.
INFINITE_LOOP_COUNT	Sonsuz çizme görevinde tamamlanması gereken döngü sayısını belirleyen yapılandırma parametresi.
Timeout	Belirli bir sürede beklenen olay gerçekleşmediğinde tetiklenen kontrol veya güvenlik mekanizması.
Fail-Safe Tetikleyicisi	Fail-safe durumunun devreye girmesine neden olan olay veya koşul.
Logger Node	Sistem olaylarını ve metrikleri kayıt altına alan yazılım bileşeni.
Simülasyon Ortamı	Gerçek uçuş öncesinde yazılımın test edildiği sanal çalışma ortamı.
H.264 / H.265	Video verisinin sıkıştırılması için kullanılan standart video codec'leri.
Streaming Modu	Kamera verisinin yer istasyonuna hangi format ve seviyede aktarılacağını belirleyen çalışma modu.
GCS (Ground Control Station)	Yer istasyonunu ifade eden genel terim.
Telemetri	İHA'dan yer istasyonuna aktarılan konum, hız, irtifa ve sistem durumu gibi uçuş verileri.
MAVLink	Uçuş bilgisayarı ile yer istasyonu veya yardımcı bilgisayar arasında kullanılan hafif haberleşme protokolü.
Algılama (Perception)	Kamera ve sensör verilerinden çevresel bilgilerin çıkarılması süreci.
Girdi / Çıktı (I/O)	Yazılım bileşenlerinin veri aldığı (girdi) ve ürettiği (çıktı) bilgileri ifade eder.
Konfigürasyon Parametresi	Yazılım davranışını çalışma anında değiştirmeye imkân tanıyan, yapılandırılabilir ayar değeri.
Operatör Müdahalesi	Yer istasyonu üzerinden, görev güvenliği veya görev sonlandırma amacıyla yapılan manuel kontrol işlemi.
Deterministik Davranış	Aynı koşullar altında sistemin her zaman aynı çıktıyı üretmesini sağlayan davranış modeli.
FSM (Finite State Machine)	Sonlu sayıda durum ve bu durumlar arasındaki geçişlerle sistem davranışını yöneten kontrol modeli.

1.4 Görev Tanımı

SAÜRO İHA sisteminin temel görevi, yarışma senaryoları kapsamında tanımlanan görevleri otonom olarak yerine getirebilen bir yazılım ve donanım bütünlüğü oluşturmaktır. Sistem, görev yürütme sürecinde insan müdahalesini minimum seviyede tutacak şekilde tasarlanmış olup, görevlerin güvenli, kontrollü ve öngörülebilir bir biçimde icra edilmesini hedeflemektedir.

Görev tanımı; kalkış öncesi hazırlık, otonom görev yürütme, hedef algılama ve etkileşim, görev tamamlanması ve güvenli uçuş sonlandırma adımlarını kapsayan bütüncül bir süreci ifade etmektedir. Bu süreç boyunca sistem, uçuş kontrolü görevlerini uçuş bilgisayarı (flight controller) üzerinden yürütürken, görev mantığı ve karar verme süreçleri yardımcı bilgisayar üzerinde çalışan yazılım tarafından yönetilmektedir.

SAÜRO İHA yazılımı, görevlerin belirli durumlar altında yürütülmesini sağlayan durum bazlı bir kontrol yaklaşımı benimsemektedir. Görev akışı, önceden tanımlanmış durumlar ve bu durumlar arasındaki geçiş koşulları çerçevesinde yönetilmekte; böylece görevlerin hem normal hem de olağan dışı senaryolarda tutarlı şekilde yürütülmesi amaçlanmaktadır.

Görev yürütme sürecinde kamera ve sensörlerden elde edilen veriler, yazılım tarafından işlenerek görev kararlarına girdi sağlamaktadır. Algılama, takip ve hedefe yaklaşma gibi işlevler, görev yönetim sistemi ile entegre şekilde çalışmakta ve yazılımın otonom karar alma yeteneğini desteklemektedir.

Sistem, görev sırasında oluşabilecek haberleşme kaybı, algılama hataları veya beklenmeyen durumlara karşı güvenli davranış sergileyecek şekilde tasarlanmıştır. Bu kapsamda, görev tanımı yalnızca ideal senaryoları değil, aynı zamanda hata ve anormal durumlar karşısında uygulanacak güvenli geçişleri de içermektedir.

Bu görev tanımı, SAÜRO İHA yazılımının tasarım ve geliştirme süreçlerine yön veren yüksek seviyeli bir çerçeve sunmakta olup, sonraki bölümlerde detaylandırılan yazılım mimarisi ve tasarım kararları için temel referans niteliği taşımaktadır.

Yazılımdan Beklenenler

Bu bölümde, SAÜRO İHA sisteminde geliştirilecek yazılımdan beklenen temel nitelikler ve davranışlar tanımlanmaktadır. Belirlenen beklentiler; görev tanımı, yazılımsal ve donanımsal ihtiyaçlar doğrultusunda, yazılım tasarımına yön verecek çerçeveyi oluşturmayı amaçlamaktadır.

Yazılımdan beklenen temel özellikler aşağıda özetlenmiştir:

01 Otonom Çalışma Kabiliyeti

Yazılım, görev yürütme sürecini insan müdahalesine minimum düzeyde ihtiyaç duyacak şekilde otonom olarak yönetebilmelidir. Görev akışı, önceden tanımlanmış durumlar ve geçiş koşulları çerçevesinde kontrol edilmelidir.

02 Durum Bazlı ve Tutarlı Davranış

Görevlerin tutarlı ve tekrarlanabilir şekilde yürütülebilmesi için yazılımın durum bazlı bir kontrol yapısı ile çalışması beklenmektedir. Bu yapı sayesinde sistem davranışları açık, izlenebilir ve öngörülebilir olmalıdır.

03 Algılama ve Karar Entegrasyonu

Kamera ve sensörlerden elde edilen verilerin, görev yürütme sürecine etkin şekilde entegre edilmesi beklenmektedir. Yazılım, algılama çıktılarından elde edilen bilgileri karar verme mekanizmalarında kullanabilmelidir.

04 Güvenli ve Fail-Safe Davranış

Yazılım, haberleşme kaybı, algılama hataları veya beklenmeyen durumlar karşısında kontrollü ve güvenli davranış sergilemelidir. Fail-safe yaklaşımlar, uçuş bilgisayarı tarafından sağlanan güvenlik mekanizmaları ile uyumlu olacak şekilde tasarlanmalıdır.

05 İzlenebilirlik ve Yer İstasyonu Desteği

Görev durumu, sistem sağlığı ve temel telemetri bilgilerinin yer istasyonu üzerinden izlenebilir olması beklenmektedir. Bu özellik, test, entegrasyon ve görev sırasında sistemin kontrol edilebilirliğini artırmalıdır.

06 Modüler ve Genişletilebilir Yapı

Yazılım, ilerleyen aşamalarda yeni görevlerin, algoritmaların veya donanım bileşenlerinin eklenmesine imkân verecek modüler bir mimariye sahip olmalıdır. Bu yapı, bakım ve geliştirme süreçlerini kolaylaştırmalıdır.

07 Simülasyon - Gerçek Uyum

Geliştirilen yazılımın, simülasyon ortamında doğrulanabilir ve gerçek sistemde uygulanabilir olması beklenmektedir. Yazılım mimarisi, simülasyondan gerçek sisteme geçişi destekleyecek şekilde tasarlanmalıdır.

08 Kaynak Verimliliği ve Zaman Duyarlılığı

Yazılımın, sınırlı işlem gücü ve bellek kaynaklarına sahip donanımlar üzerinde çalışacağı göz önünde bulundurularak, kaynakları verimli kullanması ve görev yürütme sürecinde zaman duyarlılığına sahip davranışlar sergilemesi beklenmektedir.

Bu beklentiler, SAÜRO İHA yazılımının tasarım ve geliştirme süreçlerinde temel referans noktalarını oluşturmaktadır. Bir sonraki bölümde sunulan yazılım tasarımı, bu beklentiler doğrultusunda şekillendirilmiştir.

Yazılımsal İhtiyaçlar

Bu bölümde, SAÜRO İHA sisteminin yazılım tarafında karşılaması gereken temel ihtiyaçlar tanımlanmaktadır. Belirlenen yazılımsal ihtiyaçlar; simülasyon ortamında edinilen deneyimler, yaygın İHA mimarileri ve yarışma gereksinimleri göz önünde bulundurularak oluşturulmuştur. Sistem, uçuş kontrolcüsü, yardımcı bilgisayar (companion computer), kamera modülü ve yer istasyonu arasındaki etkileşimi kapsayacak şekilde ele alınmıştır.

01

2.1.1

Uçuş Kontrolü ve Yazılım Entegrasyonu

Yazılım, uçuş kontrolcüsü ile çift yönlü haberleşme kurabilecek şekilde tasarlanmalıdır. Bu kapsamda uçuş modları, konum bilgisi, hız, irtifa ve görev durumları gibi kritik verileri alabilmesi ve gerekli durumlarda uçuş kontrolcüsüne komut gönderebilmesi gerekmektedir.

Bu etkileşim, yaygın olarak kullanılan MAVLink haberleşme protokolü üzerinden sağlanmalı ve yazılım mimarisi bu protokolün sunduğu mesaj yapıları ile uyumlu olacak şekilde kurgulanmalıdır. Simülasyon ortamında Ardupilot SITL kullanılarak gerçekleştirilen çalışmalar, bu yapının doğrulanmasına olanak sağlamaktadır.

02

2.1.2

Görev Yönetimi ve Durum Bazlı Kontrol İhtiyacı

Yazılımın, yarışma görevlerini otonom olarak yürütebilmesi için görev akışını yöneten bir kontrol mekanizmasına sahip olması gerekmektedir. Bu ihtiyaç doğrultusunda, görev yürütme sürecinin durum makinesi (state machine) tabanlı bir yapı ile yönetilmesi öngörülmektedir.

Durum bazlı kontrol yapısı sayesinde:

- Görev adımları belirli durumlar altında yürütülebilir,
- Geçiş koşulları açık şekilde tanımlanabilir,
- Hata ve olağan dışı durumlarda güvenli durumlara geçiş sağlanabilir.

Bu yaklaşım, hem simülasyon ortamında hem de gerçek sistemde tekrar edilebilir ve öngörülebilir bir görev yürütme süreci sağlamaktadır.

03

2.1.3

Görüntü İşleme ve Kamera Verisi Kullanımı

Sistem, yardımcı bilgisayar (Raspberry Pi) üzerine bağlı kamera modülünden elde edilen görüntü verisini işleyebilecek bir yazılım altyapısına ihtiyaç duymaktadır. Kamera verisinin, görev yürütme sürecine girdi sağlayacak şekilde kullanılabilmesi temel bir yazılımsal gereksinimdir.

Bu kapsamda yazılım:

- Kameradan gelen ham görüntü verisini alabilmeli,
- Görüntü işleme modüllerine iletebilmeli,
- İşlenen verileri görev yönetim sistemi ile paylaşabilmelidir.

Simülasyon ortamında ROS 2, Gazebo ve OpenCV tabanlı kamera akışları kullanılarak bu ihtiyacın doğrulanması mümkün olmaktadır.

04

2.1.4

Yer İstasyonu ile Haberleşme İhtiyacı

Yazılım, yer istasyonu ile çift yönlü haberleşme kurabilecek şekilde tasarlanmalıdır. Bu haberleşme sayesinde:

- Uçuş ve görev durumu izlenebilmeli,
- Telemetri verileri yer istasyonuna aktarılabilmesi,
- Gerekli durumlarda operatör müdahalesi mümkün olmalıdır.

Yer istasyonu ile iletişim, doğrudan uçuş kontrolcüsü üzerinden veya yardımcı bilgisayar aracılığıyla gerçekleştirilebilecek esnek bir yapı ile ele alınmalıdır. Bu yaklaşım, test ve entegrasyon süreçlerinde sistemin izlenebilirliğini artırmaktadır.

05

2.1.5

Güvenlik ve Fail-Safe Davranış İhtiyacı

Yazılım, olası hata ve anormal durumlarda sistemin güvenli davranış sergilemesini sağlayacak mekanizmalara sahip olmalıdır. Bu kapsamda yazılımsal ihtiyaçlar arasında:

- Haberleşme kaybı,
- Görev yürütme hataları,
- Sensör veya kamera verisi kaybı
- gibi durumların algılanabilmesi ve bu durumlara uygun tepkilerin üretilebilmesi yer almaktadır.

Fail-safe yaklaşımlar, uçuş kontrolcüsü seviyesindeki güvenlik önlemleri ile uyumlu olacak şekilde yazılım tasarımına entegre edilmelidir.

06

2.1.6

Modüler ve Genişletilebilir Yazılım Yapısı

Yazılımın, ilerleyen aşamalarda yeni görevlerin, algoritmaların veya sensörlerin eklenmesine imkân verecek şekilde modüler bir yapıda tasarlanması temel bir yazılımsal ihtiyaçtır. Modüler yapı sayesinde:

- Geliştirme ve bakım süreçleri kolaylaşır,
- Simülasyon ve gerçek sistem arasında geçiş daha sorunsuz gerçekleşir,
- Sistem bileşenleri bağımsız olarak test edilebilir.

Bu yaklaşım, simülasyon ortamında geliştirilen yazılım bileşenlerinin gerçek sisteme uyarlanabilirliğini desteklemektedir.



Tasarım Öncesi Not

Bu bölümde ele alınan yazılımsal ihtiyaçlar, doğrudan bir çözüm veya uygulama sunmaktan ziyade, yazılım tasarımının hangi gereksinimler doğrultusunda şekillendirileceğini tanımlamayı amaçlamaktadır. Bir sonraki bölümde yer alan tasarım kararları, bu ihtiyaçlar temel alınarak oluşturulmuştur.

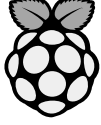


Simülasyon ↔ Gerçek Sistem Bağlantısı

Yazılımsal ihtiyaçlar belirlenirken, simülasyon ortamında edinilen deneyimler temel alınmış; gerçek sistemde karşılaşılabilecek kısıtlar ve yaygın İHA yazılım mimarileri göz önünde bulundurulmuştur. Bu yaklaşım, geliştirilecek yazılımın simülasyon ortamından gerçek sisteme uyarlanabilirliğini artırmayı hedeflemektedir.

Donanımsal ihtiyaçlar

Bu bölümde, SAÜRO İHA yazılımının çalışabilmesi için gerekli olan temel donanımsal bileşenler ve bu bileşenlere yönelik gereksinimler tanımlanmaktadır. Donanımsal ihtiyaçlar belirlenirken, yazılımsal gereksinimler, simülasyon ortamında edinilen deneyimler ve yaygın İHA sistem mimarileri göz önünde bulundurulmuştur.



RaspberryPi YARDIMCI BİLGİSAYAR

Sistem, görüntü işleme ve görev yönetimi gibi hesaplama gerektiren işlemleri gerçekleştirebilmek amacıyla bir yardımcı bilgisayara ihtiyaç duymaktadır. Bu kapsamda **Raspberry Pi 4B**, işlem gücü, enerji tüketimi ve ekosistem desteği açısından görev gereksinimlerini karşılayabilecek yeterlilikte bir platform olarak değerlendirilmiştir.

Raspberry Pi 4B'nin; kamera verilerinin işlenmesi, uçuş kontrolcüsü ile haberleşme ve yer istasyonu ile veri paylaşımı gibi görevleri eş zamanlı olarak yürütebilmesi temel bir donanımsal gereksinimdir.



PIXHAWK UÇUŞ KONTROLCÜSÜ

Sistem, uçuş kontrolü ve düşük seviye stabilizasyon görevleri için bir uçuş kontrolcüsüne ihtiyaç duymaktadır. Bu kapsamda **Pixhawk The Cube Orange**, işlem gücü, sensör redundansı ve otonom görev yetenekleri açısından tercih edilen platformdur.

Önceki nesil kontrolcülerin (ör. Pixhawk 2.4.8) işlem gücü ve genişletilebilirlik açısından ilerleyen aşamalarda yetersiz kalma riski bulunduğundan, daha güçlü ve endüstriyel seviyeye yakın bir uçuş kontrolcüsü seçilmesi uygun görülmüştür.



RaspberryPi KAMERA SİSTEMLERİ

Yazılımın görevleri yerine getirebilmesi için birden fazla kamera kaynağından veri alabilmesi gerekmektedir. Bu doğrultuda sistemde iki farklı kamera kullanım senaryosu öngörülmüştür:

Raspberry Pi Camera Module 2:

- Alt kamera olarak kullanılması planlanan bu kamera, bırakma/alma görevlerinde hedef alanın hassas şekilde algılanabilmesi ve ince ayar yapılabilmesi amacıyla tercih edilmiştir.

Raspberry Pi Global Shutter Kamera:

- Ön kamera olarak konumlandırılması planlanan bu kamera, hareketli sahnelerde daha temiz ve bozulmasız görüntü elde edilmesini sağlayarak uzaktan hedef tespiti ve yaklaşma görevlerini desteklemektedir. Global shutter yapısı ve daha yüksek görüntü kalitesi, bu kameranın tercih edilmesindeki temel etkidir.



HABERLEŞME

Sistem, uçuş sırasında yer istasyonu ile veri alışverişi yapabilmek için bir haberleşme altyapısına ihtiyaç duymaktadır. Bu haberleşmenin, telemetri verilerinin aktarımı ve sistem durumunun izlenebilmesini sağlayacak şekilde çift yönlü olması gerekmektedir.

Haberleşme altyapısının, test ve entegrasyon süreçlerinde esneklik sağlayacak biçimde Wi-Fi veya telemetri tabanlı çözümlerle uyumlu olması hedeflenmektedir. Nihai haberleşme yöntemi, menzil, gecikme ve saha koşulları göz önünde bulundurularak sistem entegrasyonu aşamasında netleştirilecektir.

DONANIMSAL YAPIYA İLİŞKİN GENEL KISITLAR

Donanımsal bileşenler seçilirken aşağıdaki kısıtlar dikkate alınmıştır:

- Ağırlık ve güç tüketimi
- Uçuş süresine etkisi
- Yardımcı bilgisayar ve uçuş kontrolcüsü arasındaki haberleşme uyumu
- Kamera sistemlerinin mekanik yerleşimi

Bu kısıtlar, yazılım tasarımının gerçek sistem koşullarında uygulanabilirliğini doğrudan etkilemektedir.

Mimari Yaklaşım

Yardımcı bilgisayar (Raspberry Pi) üzerinde çalışan yazılım, ROS 2 düğümleri (nodes) ve aralarındaki topic/service/action iletişimi üzerine kuruludur. Bu yapı; algılama, görev yönetimi, uçuş bilgisayarı arayüzü ve yer istasyonu arayüzünü birbirinden bağımsız geliştirilebilir/ test edilebilir modüller haline getirir.

Aşağıda verilen örnek ROS2 haritası, 3.2'de detaylandıracağımız yapının bir "üstten görünümü"dür

ÖNERİLEN ROS 2 NODE HARİTASI	NODE	FONKSİYON
	mission_manager_node	Durum makinesini (State Machine) yürütür, görev adımlarını yönetir.
	perception_front_node	Uzak hedef tespiti/izleme çıktıları üretir.
	perception_down_node	bırakma/alma hizalama ve ince ayar çıktıları üretir.
	fc_interface_node	Pixhawk ile MAVLink köprüsü: telemetri alır ve komut gönderimini sağlar.
	safety_monitor_node	time-out, veri kaybı, iletişim kesilmesi gibi durumları izler ve fail-safe tetikler.
	gcs_interface_node	Yer istasyonuna telemetri sağlar ve görev durumu yayınlar; komut alır.
	logger_node	Olayları ve metrikleri kayıt altına alır.

ÖNERİLEN TOPIC/ARAYÜZ YAPISI	TOPIC	ARAYÜZ
	/perception/front/targets	Mission Manager'a hedef bilgisi
	/perception/down/landing_hint	Hassas hizalama ve bırakma bilgisi
	/fc/telemetry	Uçuş telemetrileri(konum, irtifa vb.)
	/mission/state	Aktif görev durumu (örn: SEARCH_TARGET)
	/mission/command	service/action → görev komutu/override
	/safety/events	Fail-Safe Tetik ve Uyarılar
	/gcs/telemetry_out & /gcs/command_in	Yer İstasyonu Mantığı

01 Yer İstasyonu İletişim Mimarisi

Yer istasyonu ile haberleşmede hibrit bir yaklaşım benimsenmiştir. Uçuşa ilişkin kritik telemetri verileri doğrudan uçuş bilgisayar (Pixhawk) üzerinden yer istasyonuna aktarılırken, kamera akışı, görüntü işleme durumu ve görev/algılama çıktıları yardımcı bilgisayar (Raspberry Pi) üzerinden yer istasyonuna iletilmektedir. Bu yapı, telemetri hattının güvenilirliğini korurken, algılama ve görev yönetimine ait zengin verilerin yer istasyonunda izlenebilmesini sağlamaktadır.

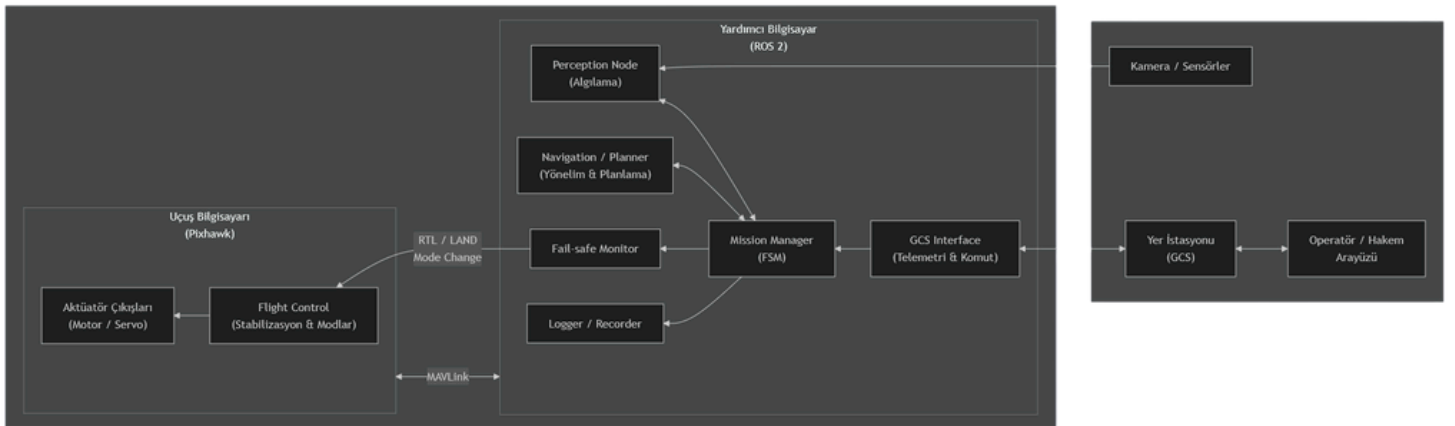


03 Genel Mimari Özeti

SAÜRO İHA yazılım mimarisi; düşük seviye uçuş kontrolü, üst seviye görev yönetimi ve algılama süreçlerinin birbirinden ayrıldığı, durum bazlı ve modüler bir yapı üzerine kuruludur.

Mimari yaklaşımda, uçuş stabilizasyonu ve emniyet-kritik işlemler uçuş bilgisayar (Pixhawk) tarafından yürütülürken; görev akışı, algılama, karar verme ve yer istasyonu etkileşimi yardımcı bilgisayar (Raspberry Pi) üzerinde çalışan ROS 2 tabanlı yazılım tarafından yönetilmektedir.

Bu ayırım, sistemin hem güvenli hem de genişletilebilir bir yapıda tasarlanmasını sağlamaktadır.

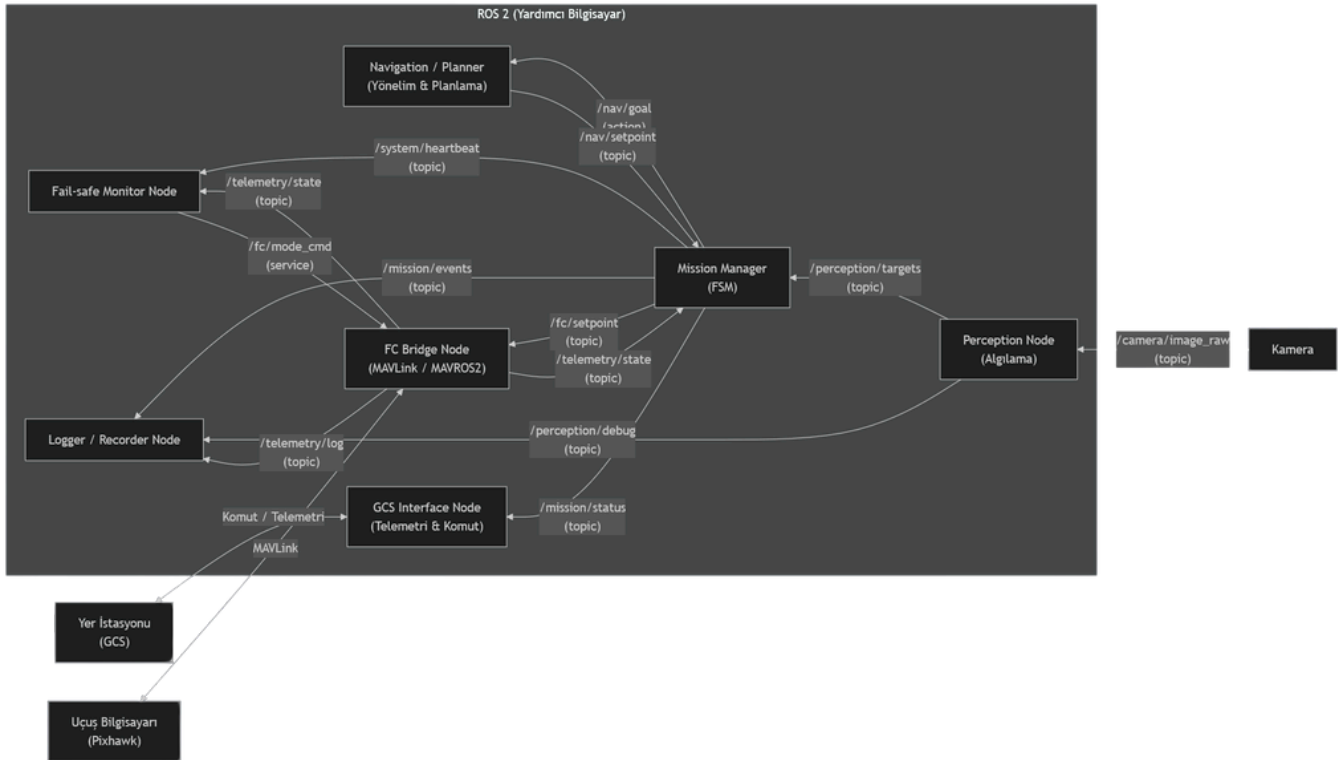
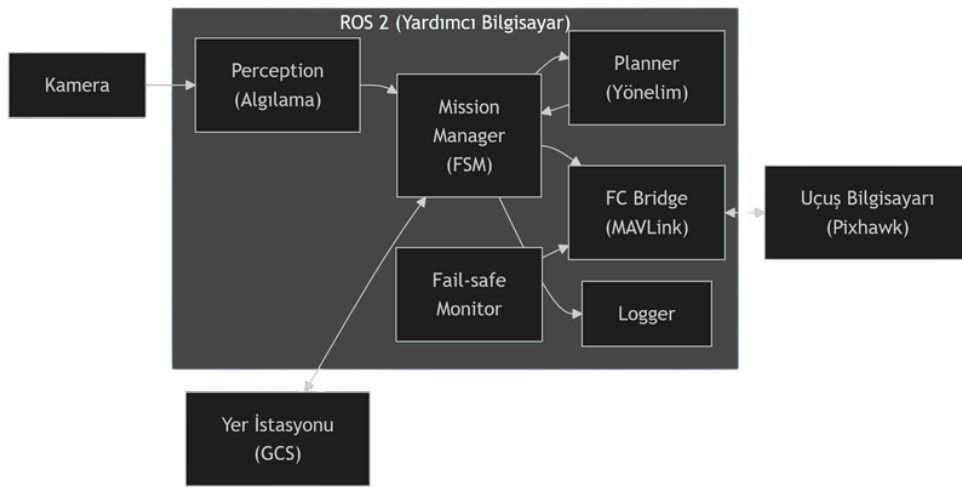


Yazılım Bileşenleri & Görevleri

Bu bölümde, yardımcı bilgisayar üzerinde ROS 2 tabanlı olarak çalışan yazılım bileşenleri ve bu bileşenlerin görevleri detaylandırılmaktadır. Tanımlanan bileşenler, 3.1 bölümünde sunulan genel mimari doğrultusunda yapılandırılmış olup, her bir bileşen belirli sorumluluklara sahip olacak şekilde tasarlanmıştır.

Yazılım bileşenleri; görev yönetimi, algılama, uçuş bilgisayarı ile etkileşim, yer istasyonu haberleşmesi ve güvenlik izleme işlevlerini kapsayan modüler bir yapı oluşturmaktadır. Yazılım bileşenleri (node'ları) ilerideki sayfada sıralanmış ve detaylandırılmıştır. Bu bileşenler veri akışı ve iletişim yapısı 3.4 bölümünde detaylandırılmaktadır.

Aşağıda ilk resim basit, 2. resim detaylı olarak yazılım bileşenlerini sunar.



01 Mission Manager Node (mission_manager_node)

Giriş

Mission Manager Node, SAÜRO İHA yazılımının merkezinde yer alan ve görev yürütme sürecini yöneten temel bileşendir. Bu node, sistemin mevcut görev durumunu takip eder ve görev akışını durum bazlı bir yapı ile yönetir.

Sorumluluklar

- Görevlerin önceden tanımlanmış durumlar çerçevesinde yürütülmesini sağlamak
- Görev durumları arasındaki geçiş koşullarını değerlendirmek
- Algılama & telemetri bileşenlerinden gelen verileri görev kararlarına girdi olarak kullanmak
- Görev ilerleyişini ve aktif durumu diğer yazılım bileşenleri ile paylaşmak

Girdiler

- Algılama çıktıları (ön ve alt kamera hedef bilgileri)
- Uçuş bilgisayarından alınan temel telemetri verileri
- Güvenlik izleme bileşeninden gelen uyarılar ve olaylar
- Yer istasyonundan alınan görev komutları (başlatma, durdurma, override vb.)

Çıktılar

- Aktif görev durumu ve görev aşaması bilgisi
- Uçuş bilgisayarına iletmek üzere üretilen görev komutları
- Yer istasyonuna iletmek üzere görev durumu özetleri

Bağımlılıklar

- Perception Front Node
- Perception Down Node
- FC Interface Node
- Safety Monitor Node
- GCS Interface Node

Dış Bağımlılıklar

- ROS2 middleware
- Node-içi State Machine

Özet

Mission Manager Node, görev yürütme sürecinin tek merkezden ve tutarlı bir şekilde yönetilmesini sağlayarak, yazılım mimarisinin öngörülebilirliğini ve kontrol edilebilirliğini artırmaktadır.

ROS 2 Topic'ler

- /perception/front/targets
- /perception/down/landing_hint
- /fc/telemetry
- /safety/events

ROS 2 Yayınları

- /mission/state

ROS 2 Service / Action

- /mission/command

02 Perception Front Node (perception_front_node)

Giriş

Perception Front Node, İHA'nın ön kamerasından elde edilen görüntü verilerini işleyerek uzaktaki hedeflerin tespit edilmesini ve görev yürütme sürecine girdi sağlayacak algılama çıktılarının üretilmesini amaçlamaktadır.

Sorumluluklar

- Ön kameradan gelen görüntü akışını almak
- Görüntü işleme algoritmalarını uygulayarak hedef tespiti gerçekleştirmek
- Tespit edilen hedeflere ait konum, tür ve güven skorlarını üretmek
- Üretilen algılama çıktılarının görev yönetimine iletilmesini sağlamak

Girdiler

- Ön kamera görüntü akışı
- Görev durum bilgisi (Mission Manager'dan gelir)

Çıktılar

- Tespit edilen hedef bilgileri (konum, sınıf, güven seviyesi)
- Algılama durum bilgisi (aktif/pasif, hata durumu vb.)

Bağımlılıklar

- Kamera sürücüleri
- OpenCV tabanlı görüntü işleme altyapısı
- Mission Manager Node

ROS 2 Topic'ler

- /camera/front/image_raw (Ön kamera ham görüntü)
- /mission/state (Aktif görev durumu)

ROS 2 Yayınları

- /perception/front/targets (Tespit edilen hedef(ler))

Dış Bağımlılıklar

- OpenCV
- Kamera Sürücüleri
- (İleride Opsiyonel: Kamera Kalibrasyon Parametreleri)

Özet

Perception Front Node, görev sürecinin erken aşamalarında algısal farkındalık sağlayarak otonom karar mekanizmalarının temel girdisini oluşturmaktadır.

03 Perception Down Node (perception_down_node)

Giriş

Perception Down Node, alt kameradan elde edilen görüntüler üzerinden hassas hizalama ve bırakma/alma görevlerine yönelik algılama çıktıları üretmekten sorumludur.

Sorumluluklar

- Alt kameradan gelen görüntü verisini almak
- Hedef alanın merkezlenmesi ve hizalama bilgilerini hesaplamak
- Bırakma veya alma görevleri için ince ayar çıktıları üretmek
- Algılama sonuçlarını görev yönetimine aktarmak

Girdiler

- Alt kamera görüntü akışı
- Aktif görev durumu ve görev aşaması bilgisi

Çıktılar

- Hassas hizalama vektörleri
- Bırakma/alma uygunluk bilgisi

Bağımlılıklar

- Kamera sürücüleri
- OpenCV tabanlı görüntü işleme altyapısı
- Mission Manager Node

ROS 2 Topic'ler

- /camera/down/image_raw (Alt kamera görüntüsü)
- /mission/state

ROS 2 Yayınları

- /perception/down/landing_hint (Merkez ofseti)

Dış Bağımlılıklar

- OpenCV
- Kamera Sürücüleri
- (İleride Opsiyonel: Kamera Kalibrasyon Parametreleri)

Özet

Perception Down Node, görevlerin kritik ve hassas aşamalarında doğruluğu artırarak görev başarımını doğrudan etkilemektedir.

04 FC Interface Node (fc_interface_node)

Giriş

FC Interface Node, yardımcı bilgisayar ile uçuş bilgisayarı (Pixhawk) arasındaki haberleşmeyi sağlayan arayüz bileşenidir.

Sorumluluklar

- MAVLink üzerinden uçuş telemetri verilerini almak
- Uçuş bilgisayarına görev ve kontrol komutlarını iletmek
- Uçuş durumunu yazılım bileşenleri ile paylaşmak

Girdiler

- Uçuş bilgisayarıdan gelen telemetri verileri
- Mission Manager tarafından üretilen kontrol ve görev komutları

Çıktılar

- Telemetri ve uçuş durumu bilgileri
- Uçuş bilgisayarına iletilen kontrol komutları

Bağımlılıklar

- MAVLink haberleşme altyapısı
- Mission Manager Node
- Safety Monitor Node

ROS 2 Topic'ler

- /mission/command (Mission Manager'dan gelen komutlar)

ROS 2 Yayınları

- /fc/telemetry (Uçuş durumu ve sensör özetleri)

Dış Arayüz

- MAVLink (UART / USB / UDP)

Dış Bağımlılıklar

- MAVLink Kütüphanesi
- Pixhawk firmware (Ardupilot)

Özet

FC Interface Node, yazılım ile uçuş donanımı arasında güvenilir bir köprü oluşturarak sistemin operasyonel bütünlüğünü sağlar..

05 Safety Monitor Node (safety_monitor_node)

Giriş

Safety Monitor Node, sistemin güvenliğini izlemek ve olası anormal durumlarda fail-safe mekanizmalarının tetiklenmesini sağlamakla sorumludur.

Sorumluluklar

- Haberleşme kesintilerini ve zaman aşımı durumlarını izlemek
- Algılama ve görev yürütme sürecindeki anormallikleri tespit etmek
- Fail-safe tetiklerini Mission Manager ve FC Interface üzerinden uygulamak

Girdiler

- Telemetry verileri
- Algılama durum bilgileri
- Görev durumu bilgisi

Çıktılar

- Fail-safe olayları ve uyarılar
- Güvenli durum geçiş talepleri

Bağımlılıklar

- FC Interface Node
- Mission Manager Node

ROS 2 Topic'ler

- /fc/telemetry
- /mission/state
- /perception/*/status

ROS 2 Yayınları

- /safety/events (Fail-safe tetik ve uyarılar yayınlanır)

Dışa Bağımlılıklar

- Zamanlayıcılar (timeout)
- Güvenlik kuralları (node-içi)

Özet

Safety Monitor Node, sistemin emniyetini koruyarak görevlerin kontrollü biçimde sonlandırılmasını garanti eder.

06 GCS Interface Node (gcs_interface_node)

Giriş

GCS Interface Node, yer istasyonu ile yazılım arasındaki haberleşmeyi sağlayarak görev durumu, algılama çıktıları ve sistem sağlığı bilgilerinin izlenmesini mümkün kılar.

Sorumluluklar

- Yer istasyonuna görev durumu ve algılama özetlerini iletmek
- Yer istasyonundan gelen komutları almak ve sisteme aktarmak
- Yer istasyonu bağlantı durumunu izlemek

Girdiler

- Görev durumu ve algılama çıktıları
- Yer istasyonundan gelen komutlar

Çıktılar

- Yer istasyonuna iletilen telemetri ve görev bilgileri
- Mission Manager'a aktarılan operatör komutları

Bağımlılıklar

- Mission Manager Node
- ROS 2 haberleşme altyapısı

ROS 2 Topic'ler

- /perception/down/landing_hint
- /mission/state
- /perception/front/targets

ROS 2 Yayınları

- /gcs/telemetry_out (Yer İstasyonuna giden veri özeti)

Dış Arayüz

- Wi-Fi (UDP / TCP veya WebSocket)

Özet

GCS Interface Node, operatör ile otonom sistem arasında kontrollü bir etkileşim katmanı oluşturmaktadır.

07 Logger Node (logger_node)

Giriş

Logger Node, görev yürütme sürecinde oluşan olayların ve kritik metriklerin kayıt altına alınmasını sağlayarak test ve analiz süreçlerini destekler.

Sorumluluklar

- Görev durum değişimlerini kayıt altına almak
- Algılama ve telemetri özetlerini loglamak
- Hata ve fail-safe olaylarını belgelemek

Girdiler

- Görev durumu bilgileri
- Algılama çıktıları
- Güvenlik olayları

Çıktılar

- Kayıt dosyaları ve analiz verileri

Bağımlılıklar

- Mission Manager Node
- Safety Monitor Node

ROS 2 Topic'ler

- /mission/state
- /fc/telemetry
- /safety/events
- /perception/*

ROS 2 Yayınları

- Pasif node, sadece gözlemler

ROS 2 Service / Action

- Pasif node, sadece gözlemler

Özet

Logger Node, sistem davranışlarının geriye dönük analizini mümkün kılarak yazılımın olgunlaşmasını destekler. Kayıt altına alınan veriler, görev sonrası analiz ve hata ayıklama amaçlı kullanılmak üzere tasarlanmıştır.

01 Durum Makinesi Tasarımı

Bu bölümde, SAÜRO İHA yazılımında görev yürütme sürecini yöneten durum makinesi yapısı tanımlanmaktadır. Durum makinesi, mission_manager_node içerisinde çalışır ve görevin her aşamasında sistemin hangi davranışı sergileyeceğini belirleyen deterministik bir kontrol mekanizması sunar. Bu yaklaşım; görev akışının öngörülebilir, izlenebilir ve tekrarlanabilir olmasını sağlar.

Durum makinesi; görev başlangıcından tamamlanmasına kadar olan normal akışın yanı sıra, anormal durumlar ve güvenlik tetikleri karşısında uygulanacak fail-safe geçişlerini de kapsayacak şekilde tasarlanmıştır.

02 Durum Makinesi Yaklaşımı

Durum makinesi tasarımında temel prensip, her bir durumun belirli bir “amaç” ve “beklenen çıktı”ya sahip olması; durumlar arası geçişlerin ise açık şekilde tanımlanmış koşullar (tetikleyiciler) altında gerçekleşmesidir. Bu yaklaşım sayesinde:

- Görev adımları belirli ve kontrol edilebilir hale gelir,
- Algılama/telemetri gibi girdilerin görev akışına etkisi şeffaflaşır,
- Zaman aşımı, veri kaybı veya güvenlik olaylarında güvenli durumlara geçiş yönetilebilir olur.

03 Fail-Safe ve Güvenli Durum Geçişleri

Durum makinesi tasarımında fail-safe yaklaşımı, yalnızca “bir hata oldu” tepkisi değil; hatanın türüne göre güvenli şekilde davranış üretilmesini hedefler. Bu nedenle fail-safe geçişleri:

- Haberleşme kaybı (Yi veya Pixhawk bağlantısı)
- Algılama kaybı (kamera/veri akışı kesilmesi)
- Zaman aşımı (belirli sürede hedef bulunamaması veya ilerleme kaydedilememesi)
- Görev sapması (rota/konum hedeflerinden aşırı sapma)

Zaman aşımı mekanizmaları, görev yürütme sürecinde ilerlemenin izlenmesi amacıyla SEARCH_TARGET, APPROACH_TARGET ve DRAW_INFINITY durumlarında uygulanır.

Fail-safe geçişleri üstte verilen olaylara karşı tetiklenebilir şekilde ele alınmıştır. Fail-safe tetiklendiğinde sistem, görev yürütme akışını durdurarak güvenli duruma geçiş sağlar ve yer istasyonu üzerinden bilgilendirme yapılır.

Fail-safe tetiklendiğinde sistem, görevi sonlandırarak güvenli uçuş sonlandırma sürecine geçer. Bu kapsamda İHA, kalkış noktasına RTL (Return-to-Launch) ile geri döner ve ardından LAND ile iniş gerçekleştirir. Fail-safe tetiklenmesi ve süreç durumu yer istasyonu üzerinden operatöre bildirilir.

04 Durum Makinesi: Temel Durumlar

Aşağıdaki durumlar, görev türünden bağımsız olarak sistemin genel görev yürütme yapısını ifade eden çekirdek durumlardır:

- **IDLE** > Sistem bekleme durumundadır. Görev henüz başlatılmamıştır.
- **ARM_TAKEOFF** > Uçuş öncesi hazırlıklar tamamlanır ve kalkış süreci başlatılır.
- **SEARCH_TARGET** > Görev için gerekli hedef(ler)in algılanması / tespit edilmesi amaçlanır.
- **APPROACH_TARGET** > Tespit edilen hedefe kontrollü yaklaşma süreci yürütülür.
- **ALIGN** > Hassas hizalama ihtiyacı olan görevlerde hedef merkezleme ve ince ayar yapılır.
- **MISSION_COMPLETE** > Görev başarıyla tamamlanmış durumdadır. Sistem güvenli şekilde görevi sonlandırmaya hazırlanır.
- **FAILSAFE** > Kritik hata veya güvenlik tetikleri sonucu sistemin güvenli davranışa geçtiği durumdur.

Not: “Bırakma/Alma” gibi görev adımları, görevin türüne göre ALIGN sonrası devreye giren alt akışlar olarak modellenenebilir (örn. DROP_PAYLOAD). Bu alt durumlar, görev kapsamı netleştikçe durum setine eklenebilir.

05 Göreve Özel Durumlar: Sonsuz Çizme

“Sonsuz çizme” gibi belirli görevler, çekirdek akışa ek olarak özel bir davranış adımı gerektirir. Bu tür görevlerde, durum makinesine görev odaklı ek durumlar dahil edilerek görev akışının okunabilirliği ve yönetilebilirliği artırılır.

Bu kapsamda, sonsuz çizme görevine özel iki durum tanımlanmıştır:

- **ENTER_INFINITY** > Sonsuz çizme görevine giriş/kurulum aşamasını temsil eder. Bu durumda sistem, çizim davranışını başlatmak için gerekli ön koşulları sağlar (mod, başlangıç noktası, güvenlik kontrolleri vb.).
- **DRAW_INFINITY** > Sonsuz çizme davranışının icra edildiği durumdur. Bu durumda sistem, çizim sürecini yürütürken sapma, zaman aşımı ve güvenlik tetiklerini izlemeye devam eder.

Sonsuz çizme görevinin tamamlanma koşulu, sistem parametresi olarak tanımlanan INFINITE_LOOP_COUNT değişkenine bağlıdır. Görev, belirtilen sayıda döngü tamamlandığında MISSION_COMPLETE durumuna geçer. Varsayılan değer INFINITE_LOOP_COUNT = 1 olarak belirlenmiştir.

INFINITE_LOOP_COUNT parametresi, sistemin varsayılan görev yapılandırması içerisinde tanımlı olup (varsayılan: 1), ihtiyaç halinde yer istasyonu üzerinden görev başlatma aşamasında override edilebilecek şekilde tasarlanmıştır. Bu yaklaşım, saha koşullarına göre görev süresinin esnek biçimde ayarlanmasını mümkün kılar.

05 Durum Geçişleri ve Tetikleyiciler

IDLE durumu durum makinesinin başlangıç durumu, MISSION_COMPLETE ise normal görev akışının son durumudur. Aşağıda temel geçiş mantığı özetlenmiştir:

Görev başlat komutu alındığında:

- **IDLE → ARM_TAKEOFF**

Kalkış başarıyla tamamlandığında ve sistem stabil hale geldiğinde:

- **ARM_TAKEOFF → SEARCH_TARGET**

Hedef tespiti güvenilir bulunduğunda:

- **SEARCH_TARGET → APPROACH_TARGET**

Hedefe yaklaşım belirli bir eşik mesafeye ulaştığında veya hassas görev aşamasına girildiğinde:

- **APPROACH_TARGET → ALIGN**

Çizim görevine giriş koşulları sağlandığında:

- **ALIGN → ENTER_INFINITY** (sonsuz çizme görevi aktifse)

Kurulum tamamlandığında ve çizim icrası başlatılabildiğinde:

- **ENTER_INFINITY → DRAW_INFINITY**

Görev başarı kriteri ($completed_loops \geq INFINITE_LOOP_COUNT$) sağlandığında veya operatör tarafından sonlandırıldığında:

- **DRAW_INFINITY → MISSION_COMPLETE**

Kritik güvenlik tetikleri (haberleşme kaybı, telemetri kaybı, algılama kaybı, zaman aşımı vb.) oluştuğunda:

- **FAILSAFE → RTL → LAND**

FAILSAFE hk. NOT: FAILSAFE durumu terminal bir durumdur; bu duruma geçildikten sonra görev yürütme akışı yeniden başlatılmadan devam etmez.

OPERATÖR hk. NOT: Operatör müdahalesi, görev yürütme sürecinde yalnızca görev sonlandırma veya güvenli duruma geçiş amacıyla kullanılmakta olup, durum makinesi içindeki otomatik geçişleri doğrudan atlayacak şekilde tasarlanmamıştır.



01 Veri Akışı ve İletişim

Bu bölümde, SAÜRO İHA yazılım bileşenleri arasında gerçekleşen veri akışı ile uçuş bilgisayar (Pixhawk), yardımcı bilgisayar (Raspberry Pi) ve yer istasyonu (Yİ) arasındaki iletişim yapısı açıklanmaktadır. Veri akışı, ROS 2 tabanlı node'lar arasındaki topic/service/action iletişimi üzerinden; harici iletişim ise MAVLink ve yer istasyonu bağlantısı üzerinden yürütülmektedir.

Bu yapı; görev yürütme sürecinin izlenebilirliğini artırmak, bileşenler arası bağımlılıkları azaltmak ve simülasyondan gerçek sisteme geçişi kolaylaştırmak amacıyla tasarlanmıştır.

02 Veri Akışı Genel Özeti

Sistem genelinde veri akışı üç ana eksenle ele alınmaktadır:

1. Algılama Verisi Akışı (Kamera → Algılama → Görev Yönetimi)
2. Uçuş Telemetrisi ve Komut Akışı (Pixhawk ↔ Pi)
3. Yer İstasyonu İletişimi (Hibrit: Pixhawk telemetri + Pi kamera/algılama)

Mission Manager Node, görev akışının merkezinde yer alır ve algılama/telemetri çıktılarından beslenerek görev kararlarını üretir.

03 Algılama Verisi Akışı

Algılama verisi akışı, ön ve alt kamera verilerinin işlenerek görev kararlarına girdi haline getirilmesini kapsar.

• Ön Kamera Akışı

- /camera/front/**image_raw** → perception_front_node
- perception_front_node → /perception/front/targets → mission_manager_node

• Alt Kamera Akışı

- /camera/down/**image_raw** → perception_down_node
- perception_down_node → /perception/down/landing_hint → mission_manager_node

Bu akışta algılama node'ları, ham görüntü verisini doğrudan diğer bileşenlere taşımak yerine, görev yönetiminde kullanılabilir "çıktı" üretir. Bu yaklaşım, veri taşınmasını azaltır ve görev karar mekanizmasını sadeleştirir.

04 Uçuş Telemetrisi Akışı

Uçuş bilgisayarından üretilen telemetri verileri, FC Interface Node üzerinden yazılım bileşenlerine aktarılır. (Pixhawk → ROS 2)

- Pixhawk telemetrisi → fc_interface_node
- fc_interface_node → /fc/telemetry → mission_manager_node, safety_monitor_node, (opsiyonel logger_node)

Telemetri verisi; konum, irtifa, hız, uçuş modu, sistem durumu gibi bilgileri içerir ve görev yürütme ile fail-safe kararlarının temel girdilerinden biridir.

05 Komut Akışı

Görev yürütme sürecinde üretilen kontrol ve görev komutları, Mission Manager Node tarafından üretilir ve Pixhawk'a FC Interface Node üzerinden iletilir. (ROS 2 → Pixhawk)

- mission_manager_node → (komut) → fc_interface_node → Pixhawk

Komutlar; görev aşamasına göre hız/irtifa hedefleri, konum hedefleri, mod değişimleri veya görev tetikleyicileri gibi kontrol çıktıları içerebilir. Komut üretimi sırasında Safety Monitor Node'un ürettiği güvenlik olayları da dikkate alınır.

06 Güvenlik Olayları ve Fail-Safe İletişimi

Safety Monitor Node, sistem sağlığını izler ve anormal durumlarda fail-safe olaylarını yayınlar:

- safety_monitor_node → /safety/events → mission_manager_node, gcs_interface_node, logger_node

Fail-safe tetiklendiğinde, durum makinesi görev yürütmeyi sonlandırır ve uçuş bilgisayarında RTL + LAND davranışı uygulanacak şekilde komut akışı devreye girer.

07 Yer İstasyonu İletişim Yapısı

Yer istasyonu ile iletişim hibrit yapıdadır:

- Uçuş telemetrisi: Pixhawk → Yİ
- Kamera / algılama / görev durumu: Raspberry Pi → Yİ

Bu yaklaşım ile kritik uçuş telemetrisi, uçuş bilgisayarından yer istasyonuna doğrudan aktarılırken; kamera çıktıları, görüntü işleme durumu ve görev durumları yardımcı bilgisayar üzerinden iletilir. Böylece telemetri hattının güvenilirliği korunurken, görev ve algılama süreçleri yer istasyonu üzerinden izlenebilir hale gelir.

08 Yer İstasyonu Veri Türleri ve Komutları

Yer istasyonuna aktarılması öngörülen temel veri türleri:

- Görev durumu (aktif state, görev ilerleyişi)
- Algılama çıktıları (hedef bilgisi, hizalama çıktıları, algılama güven seviyesi)
- Sistem sağlık bilgisi (uyarılar, fail-safe olayları)

Yer istasyonundan alınabilecek temel komutlar:

- Görev başlatma / durdurma
- Görev parametre override (örn. INFINITE_LOOP_COUNT)
- Operatör müdahalesi (güvenli sonlandırma)

Yer istasyonundan gelen komutlar, GCS Interface Node üzerinden Mission Manager Node'a iletilir ve görev akışı içinde kontrollü şekilde uygulanır.

09 Simülasyon ile Gerçek Sistem Uyumu

Veri akışı ve iletişim yaklaşımı, simülasyon ortamında test edilerek doğrulanabilir olacak şekilde kurgulanmıştır. ROS 2 tabanlı arayüzler ve MAVLink köprüsü sayesinde, simülasyonda kullanılan bileşenlerin gerçek sisteme aktarımında minimum değişiklik hedeflenmektedir.

10 Kamera Veri Akışı Politikası (Stream Modları)

Yer istasyonu ile yardımcı bilgisayar (Raspberry Pi) arasındaki kamera veri akışı, görev sırasında performans kaybını minimize etmek ve test/entegrasyon süreçlerinde gözlemlenebilirliği artırmak amacıyla seçilebilir modlar üzerinden yönetilmektedir. Bu kapsamda kamera akışı, çalışma anında değiştirilebilen bir “streaming modu” mekanizması ile kontrol edilir.

Streaming Modları

Sistem, aşağıdaki dört moddan biri ile çalışır:

- Akış Yok (**Max Performans**):

Yer istasyonuna kamera akışı gönderilmez. Sadece görev yürütme için gerekli yerel algılama çalışır.

- Sadece İşlenmiş Çıktılar (**Düşük Performans Kaybı**):

Yer istasyonuna ham görüntü yerine, algılama çıktıları (hedef bilgisi, durum, güven skoru, hizalama çıktıları vb.) iletilir.

- Sıkıştırılmış Canlı Akış (H.264 / H.265, **FPS Ayarlı**):

Kamera görüntüsü, belirlenen FPS ve bit-rate değerleri ile sıkıştırılarak yer istasyonuna aktarılır. Bu mod, özellikle saha testlerinde görsel doğrulama amacıyla kullanılır.

- Raw Image (**Yüksek Performans Kaybı**):

Kamera görüntüsü ham/işlenmemiş biçimde yer istasyonuna aktarılır. Bu mod, yalnızca kısa süreli debug ve kalibrasyon amaçlı kullanılır.

Mod Seçimi ve Çalışma Prensipleri

Streaming modu, yer istasyonu üzerinden operatör komutu ile değiştirilebilir. Varsayılan çalışma modu, görev performansını önceliklendirmek amacıyla Mod 1 veya Mod 2 olarak belirlenir. Mod 3 ve Mod 4, yalnızca test ve entegrasyon senaryolarında geçici olarak etkinleştirilir.

10 Kamera Veri Akışı Politikası (Stream Modları)

Yer istasyonu ile yardımcı bilgisayar (Raspberry Pi) arasındaki kamera veri akışı, görev sırasında performans kaybını minimize etmek ve test/entegrasyon süreçlerinde gözlemlenebilirliği artırmak amacıyla seçilebilir modlar üzerinden yönetilmektedir. Bu kapsamda kamera akışı, çalışma anında değiştirilebilen bir “streaming modu” mekanizması ile kontrol edilir.

Streaming Modları

Sistem, aşağıdaki dört moddan biri ile çalışır:

- Akış Yok (**Max Performans**):

Yer istasyonuna kamera akışı gönderilmez. Sadece görev yürütme için gerekli yerel algılama çalışır.

- Sadece İşlenmiş Çıktılar (**Düşük Performans Kaybı**):

Yer istasyonuna ham görüntü yerine, algılama çıktıları (hedef bilgisi, durum, güven skoru, hizalama çıktıları vb.) iletilir.

- Sıkıştırılmış Canlı Akış (H.264 / H.265, **FPS Ayarlı**):

Kamera görüntüsü, belirlenen FPS ve bit-rate değerleri ile sıkıştırılarak yer istasyonuna aktarılır. Bu mod, özellikle saha testlerinde görsel doğrulama amacıyla kullanılır.

- Raw Image (**Yüksek Performans Kaybı**):

Kamera görüntüsü ham/işlenmemiş biçimde yer istasyonuna aktarılır. Bu mod, yalnızca kısa süreli debug ve kalibrasyon amaçlı kullanılır.

Mod Seçimi ve Çalışma Prensibi

Streaming modu, yer istasyonu üzerinden operatör komutu ile değiştirilebilir. Varsayılan çalışma modu, görev performansını önceliklendirmek amacıyla Mod 1 veya Mod 2 olarak belirlenir. Mod 3 ve Mod 4, yalnızca test ve entegrasyon senaryolarında geçici olarak etkinleştirilir.

10.1 Varsayılan Davranış ve Yapılandırma

Sistem, görev performansını önceliklendirmek amacıyla varsayılan olarak Mod 1 – Akış Yok durumunda çalışacak şekilde yapılandırılmıştır. Varsayılan streaming modu, konfigürasyon dosyası üzerinden değiştirilebilir olup, görev başlatma öncesinde ihtiyaca göre ayarlanabilir.

Streaming modları, görev yürütme sırasında yer istasyonu üzerinden operatör komutu ile değiştirilebilir.

10.2 Arayüz ve Kontrol Mekanizması

Kamera streaming modu, yer istasyonu üzerinden gönderilen kontrol komutları ile yönetilir. Önerilen kontrol arayüzü aşağıdaki şekildedir:

Streaming Modu Ayarı:

- SET_STREAM_MODE(mode, fps, bitrate)
 - mode $\in \{1, 2, 3, 4\}$
 - fps yalnızca Mod 3 ve Mod 4 için kullanılır
 - bitrate yalnızca Mod 3 için kullanılır

Yer istasyonundan gelen streaming komutları, GCS Interface Node üzerinden ilgili algılama ve kamera bileşenlerine iletilir. Streaming modu değişiklikleri, görev durumunu veya durum makinesini doğrudan etkilemez.

10.3 Otomatik Mod Düşürme Mekanizması

Otomatik mod düşürme mekanizması, yalnızca kamera veri akışı modlarını etkileyecek şekilde tasarlanmıştır. Bu mekanizma kapsamında:

- Mod 3 veya Mod 4 aktifken
- CPU yükü, gecikme veya bağlantı kalitesi belirli eşiklerin altına düştüğünde

sistem otomatik olarak Mod 2 – Sadece İşlenmiş Çıktılar moduna geçiş yapar. Bu geçiş:

- Görev yürütme akışını durdurmaz
- Durum makinesini etkilemez
- Operatöre yer istasyonu üzerinden bildirilir

10.4 Güvenlik ve Performans Önceliği

Kamera veri akışı, görev güvenliğini ve uçuş emniyetini etkilemeyecek şekilde sınırlandırılmıştır. Yüksek bant genişliği veya işlem gücü gerektiren streaming modları (Mod 3 ve Mod 4) etkinleştirildiğinde, sistem kaynak kullanımı izlenir.

Kaynak kullanımının görev yürütme sürecini olumsuz etkilemesi durumunda, sistem otomatik olarak daha düşük seviyeli bir streaming moduna geçiş yapabilir.

11 Bölüm Özeti

Bu bölümde, SAÜRO İHA yazılımında veri akışı ve iletişim yapısı detaylı şekilde ele alınmıştır. Algılama verileri, uçuş telemetrisi ve yer istasyonu iletişimi; görev yönetimini merkeze alan, modüler ve izlenebilir bir yapı içerisinde tanımlanmıştır. Özellikle kamera veri akışı için belirlenen streaming modları ve otomatik mod düşürme mekanizması, görev güvenliğini ve sistem performansını önceliklendiren esnek bir iletişim yaklaşımı sunmaktadır. Tanımlanan veri akışı ve iletişim mimarisi, simülasyon ortamından gerçek sisteme geçişi destekleyecek şekilde kurgulanmış olup, sonraki bölümlerde ele alınan güvenlik ve yapılandırma kararları için temel teşkil etmektedir.

01 Güvenlik ve Performans Önceliği

Bu bölümde, SAÜRO İHA yazılımında görev yürütme sürecinin güvenliğini sağlamak amacıyla benimsenen fail-safe yaklaşımı ve güvenlik mekanizmaları açıklanmaktadır. Güvenlik tasarımı, önceki bölümlerde tanımlanan mimari, yazılım bileşenleri, durum makinesi ve veri akışı kararları ile uyumlu olacak şekilde ele alınmıştır.

Sistem, normal görev akışının yanı sıra; haberleşme, algılama veya uçuşla ilgili anormal durumlarda güvenli davranış üretmeyi önceliklendirecek şekilde tasarlanmıştır.

02 Güvenlik Yaklaşımı

SAÜRO İHA yazılımında güvenlik yaklaşımı, görev başarısından önce uçuş emniyetini ve sistem bütünlüğünü korumayı hedefler. Bu kapsamda:

- Güvenlik kararları görev mantığından ayrıştırılmıştır
- Kritik tetikleyiciler merkezi olarak izlenmektedir
- Fail-safe davranışı deterministik ve öngörülebilir şekilde tanımlanmıştır
- Yer istasyonu üzerinden gönderilen komutlar, görev güvenliğini ihlal etmeyecek şekilde yazılım tarafından doğrulanır.

Bu yaklaşım sayesinde, görev yürütme sırasında oluşabilecek beklenmeyen durumlar kontrol altına alınabilmektedir.

03 Fail-Safe Durumunun Kapsamı

FAILSAFE durumu, terminal bir durum olarak ele alınmıştır. Bu duruma geçildikten sonra görev yürütme akışı otomatik olarak devam etmez ve sistem, yeni bir görev başlatılana kadar güvenli bekleme durumunda kalır.

Bu tasarım kararı, görev akışının kontrolsüz biçimde yeniden başlamasını engellemeyi ve operatör kontrolünü önceliklendirmeyi amaçlamaktadır.

04 Fail-Safe Tetikleyicileri

Fail-safe mekanizması, aşağıdaki durumların herhangi birinin gerçekleşmesi halinde tetiklenir:

- **Haberleşme Kaybı**

Yer istasyonu, yardımcı bilgisayar veya uçuş bilgisayarı arasındaki iletişimin kaybedilmesi

- **Algılama Kaybı**

Kamera verisinin veya algılama çıktılarının belirli bir süre boyunca alınamaması

- **Zaman Aşımı**

Görev ilerleyişinin belirli durumlarda (SEARCH_TARGET, APPROACH_TARGET, DRAW_INFINITY) tanımlı süreler içerisinde gerçekleşmemesi

- **Görev Sapması**

Hedef konum, rota veya görev kriterlerinden kabul edilebilir eşiklerin üzerinde sapma oluşması

- **Sistem Sağlık Problemleri**

Aşırı kaynak kullanımı veya kritik yazılım/hardware hataları

05 Fail-Safe Davranışı

Fail-safe tetiklendiğinde sistem, mevcut görev yürütme akışını sonlandırarak güvenli uçuş sonlandırma sürecine geçer. Bu kapsamda:

1. Görev yürütme durdurulur
2. Durum makinesi FAILSAFE durumuna geçer
3. Uçuş bilgisayarı RTL (Return-to-Launch) modu tetiklenir
4. Kalkış noktasına dönüş tamamlandıktan sonra LAND ile iniş gerçekleştirilir

Fail-safe süreci boyunca sistem durumu ve tetikleyici nedeni yer istasyonu üzerinden operatöre bildirilir.

06 Güvenlik Mekanizmalarının diğ er Bileş enlerle Etkileş imi

Güvenlik ve fail-safe mekanizmaları, sistemin diğ er yazılım bileş enleri ile aş ğıdaki şekilde etkileş im iç erisindedir:

- **Mission Manager Node**

Fail-safe tetiklerini görev durumuna yansıtır ve görev akış ını sonlandırır

- **Safety Monitor Node**

Sistem sağ lıđ ını izler ve fail-safe olaylarını üretir

- **FC Interface Node**

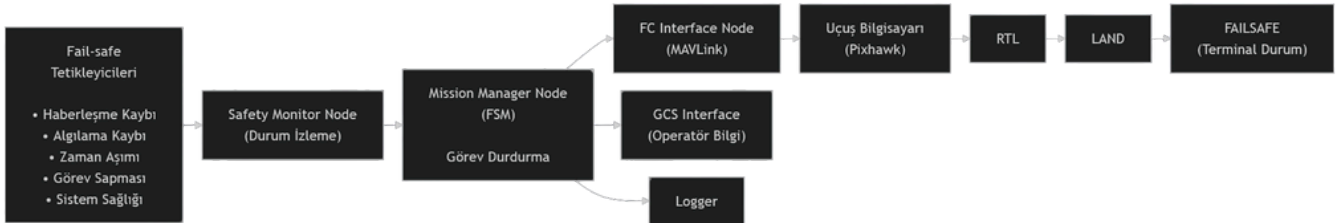
Fail-safe durumunda uç uş bilgisayarına gerekli RTL ve LAND komutlarını iletir

- **GCS Interface Node**

Operatör bilgilendirmesini sağ lar

Bu yapı sayesinde güvenlik kararları merkezi, ancak uygulama dağı tık bir mimari ile gerçekleştirilir.

ÖRNEK FAIL-SAFE DİY AGRAMI:



Bu akış , görev yazılım ı tarafından tetiklenen fail-safe (otonom güvenli sonlandırma) mekanizmasını ifade etmektedir. Radyo haberleş mesine bağı lı fail-safe davranış ları, uç uş kontrolcüsü (Pixhawk) konfigürasyonu kapsamında ele alınmakta ve bu diyagramın kapsamı dış ında tutulmaktadır.

07 Bölüm Özeti

Bu bölümde, SAÜRO İHA yazılım ında güvenlik ve fail-safe tasarımı ele alınmıştır. Tanımlanan tetikleyiciler ve fail-safe davranış ı, görev yürütme sürecinden bağı msız ancak onunla uyumlu olacak şekilde yapılandırılmış tır. Bu yaklaş ım, sistemin anormal durumlar karş ısında kontrollü ve öngörülebilir biçimde davranmasını sağlayarak uç uş güvenliđ ini önceliklendirmektedir.

01 Konfigürasyon ve Parametre Yönetimi

Bu bölümde, SAÜRO İHA yazılımında görev yürütme, kamera veri akışı ve güvenlik mekanizmalarına ait yapılandırılabilir parametreler ile bu parametrelerin yönetim yaklaşımı açıklanmaktadır. Konfigürasyon yönetimi, sistemin farklı görev senaryolarına ve saha koşullarına uyarlanabilmesini sağlamak amacıyla tasarlanmıştır.

02 Konfigürasyon Yaklaşımı

Sistemde kullanılan yapılandırma parametreleri, görevden bağımsız çalışabilecek şekilde merkezi bir konfigürasyon yapısı altında toplanmıştır. Varsayılan değerler yazılım konfigürasyonu içerisinde tanımlanmış olup, ihtiyaç halinde görev başlatma aşamasında yer istasyonu üzerinden override edilebilecek şekilde tasarlanmıştır.

Bu yaklaşım sayesinde:

- Yazılım yeniden derlenmeden görev davranışı değiştirilebilir
- Test ve saha senaryoları arasında hızlı geçiş sağlanabilir
- Görev güvenliği korunurken esneklik artırılır

03 Görev Parametreleri

Görev yürütme sürecini doğrudan etkileyen temel parametreler aşağıda listelenmiştir:

• INFINITE_LOOP_COUNT

Sonsuz çizme görevinde tamamlanması gereken döngü sayısını belirler.

Varsayılan değer: **1**

Yer istasyonu üzerinden görev başlatılırken override edilebilir.

• MISSION_TIMEOUTS

Belirli görev durumları için tanımlanan zaman aşımı değerlerini içerir.

Bu parametreler, SEARCH_TARGET, APPROACH_TARGET ve DRAW_INFINITY durumlarında ilerlemenin izlenmesi amacıyla kullanılır.

04 Kamera ve Streaming Parametreleri

Kamera veri akışı politikası kapsamında kullanılan temel yapılandırma parametreleri aşağıda verilmiştir:

- **DEFAULT_STREAM_MODE**

Sistem açılışında kullanılacak varsayılan kamera streaming modunu belirler. Varsayılan değer: Mod 1 – Akış Yok

- **STREAM_FPS**

Sıkıştırılmış canlı akış modunda (Mod 3) kullanılacak kare hızını belirler.

- **STREAM_BITRATE**

Sıkıştırılmış canlı akış modunda (Mod 3) kullanılacak bit-rate değerini belirler.

- **AUTO_STREAM_DOWNGRADE**

Otomatik mod düşürme mekanizmasının aktif olup olmayacağını belirler. Varsayılan: aktif

Bu parametreler, görev güvenliği ve sistem performansı göz önünde bulundurularak yapılandırılabilir.

05 Güvenlik ve Fail-Safe Parametreleri

Fail-safe mekanizmasına ait yapılandırılabilir parametreler aşağıdaki gibidir:

- **COMMUNICATION_TIMEOUT**

Yer istasyonu, yardımcı bilgisayar veya uçuş bilgisayarı arasındaki iletişim kaybı için tanımlanan eşik süresi.

- **PERCEPTION_TIMEOUT**

Algılama çıktılarının belirli bir süre boyunca alınamaması durumunda fail-safe tetiklenmesini sağlayan süre.

- **RESOURCE_USAGE_LIMITS**

CPU ve sistem kaynak kullanımına ilişkin eşik değerleri. Bu eşikler aşıldığında otomatik mod düşürme veya fail-safe mekanizmaları devreye girebilir.

Zaman aşımı parametreleri, sistem saatine bağlı olarak saniye cinsinden tanımlanır.

06 Parametre Güncelleme ve Yetkilendirme

Parametre güncellemeleri, yalnızca yer istasyonu üzerinden ve görev başlatma aşamasında veya görev yürütme sırasında tanımlı kurallar çerçevesinde gerçekleştirilebilir. Görev güvenliğini etkileyebilecek kritik parametreler, görev yürütme sırasında sınırlı veya kontrollü şekilde değiştirilebilir.

Bu yaklaşım, operatör hatalarının ve kontrolsüz yapılandırma değişikliklerinin önüne geçmeyi amaçlamaktadır.

07 Bölüm Özeti

Bu bölümde, SAÜRO İHA yazılımında kullanılan konfigürasyon ve parametre yönetimi yaklaşımı ele alınmıştır. Tanımlanan parametreler; görev davranışlarının, kamera veri akışı politikasının ve güvenlik mekanizmalarının esnek ancak kontrollü biçimde yönetilmesini sağlamaktadır. Bu yapı, farklı görev senaryoları ve test koşulları için yazılımın yeniden derlenmesine gerek kalmadan uyarlanabilmesini mümkün kılmaktadır.

01 Test ve Doğrulama Yaklaşımı

Bu bölümde, SAÜRO İHA yazılımının işlevsel doğruluğunu, güvenliğini ve performansını doğrulamak amacıyla izlenen test yaklaşımı açıklanmaktadır. Test ve doğrulama süreci; simülasyon ortamından gerçek sisteme geçişi destekleyecek şekilde kademeli ve senaryo bazlı olarak ele alınmıştır.

Amaç, görev yürütme mantığının, veri akışının ve fail-safe mekanizmalarının tasarımı öngörüldüğü şekilde çalıştığını doğrulamaktır.

02 Test Yaklaşımı ve Kapsam

Test süreci aşağıdaki temel prensiplere dayanmaktadır:

- Bileşenlerin önce tekil, ardından entegre şekilde test edilmesi
- Simülasyon ortamında doğrulanan davranışların gerçek sistemde tekrar edilmesi
- Normal görev akışı kadar anormal durumların da test edilmesi
- Güvenlik ve fail-safe davranışlarının kontrollü şekilde tetiklenmesi

Bu kapsamda testler; yazılım bileşenleri, durum makinesi, veri akışı ve güvenlik mekanizmalarını kapsayacak şekilde planlanmıştır.

03 Simülasyon ve Gerçek Sistem Testleri

Test süreci iki ana aşamada yürütülür:

- Simülasyon Ortamı
 - ROS 2 tabanlı simülasyon
 - Görev senaryolarının hızlı ve güvenli test edilmesi
- Gerçek Sistem
 - Donanım entegrasyonunun doğrulanması
 - Gerçek sensör ve haberleşme koşullarında test

Simülasyon ortamında doğrulanan davranışların, gerçek sistemde minimum değişikliklerle tekrar edilebilmesi hedeflenmektedir.

04 Bileşen Bazlı Testler

3.7.2 Bileşen Bazlı Testler

Her yazılım bileşeni, kendi sorumlulukları doğrultusunda izole olarak test edilir:

- **Algılama Node'ları (Front / Down)**
 - Farklı ışık koşulları ve hedef senaryoları
 - Algılama çıktılarının doğruluğu ve sürekliliği
- **Mission Manager Node**
 - Durum geçişlerinin doğru tetiklenmesi
 - Görev akışının deterministik şekilde ilerlemesi
- **FC Interface Node**
 - Telemetri verisinin doğru alınması
 - Komutların uçuş bilgisayarına doğru iletilmesi
- **GCS Interface Node**
 - Yer istasyonu komutlarının doğru işlenmesi
 - Görev ve durum bilgilerinin doğru iletilmesi
- **Safety Monitor Node**
 - Güvenlik tetikleyicilerinin doğru algılanması
 - Fail-safe olaylarının doğru üretilmesi

05 Durum Makinesi Doğrulama Testleri

Durum makinesi, görev senaryoları üzerinden test edilir. Bu kapsamda:

- Normal görev akışı (IDLE → MISSION_COMPLETE)
- Göreve özel durumlar (ENTER_INFINITY → DRAW_INFINITY)
- Zaman aşımı ve hata senaryoları
- FAILSAFE tetiklenmesi ve terminal davranış

kontrollü olarak test edilir. Her test senaryosunda, beklenen durum geçişleri ve gerçekleşen durumlar karşılaştırılarak doğrulama yapılır.

06 Veri Akışı ve İletişim Testleri

Veri akışı testleri, bileşenler arası iletişimin ve yer istasyonu bağlantısının doğrulanmasını hedefler:

- Algılama çıktılarının Mission Manager'a doğru iletilmesi
- Telemetri ve komut akışının sürekliliği
- Kamera streaming modlarının doğru çalışması
- Streaming modları arasında geçişlerin doğrulanması

Bu testlerde özellikle Mod 1–4 kamera veri akışı modları ve otomatik mod düşürme mekanizması gözlemlenir.

07 Güvenlik ve Fail-Safe Testleri

Fail-safe testleri, sistemin anormal durumlar karşısında güvenli davranış ürettiğini doğrulamak amacıyla gerçekleştirilir:

- Haberleşme kaybı senaryoları
- Algılama kaybı senaryoları
- Zaman aşımı tetiklemeleri
- Yüksek kaynak kullanımı senaryoları

Fail-safe tetiklendiğinde sistemin RTL + LAND davranışını doğru şekilde uyguladığı doğrulanır.

08 Bölüm Özeti

Bu bölümde, SAÜRO İHA yazılımı için benimsenen test ve doğrulama yaklaşımı ele alınmıştır. Tanımlanan test senaryoları; yazılım bileşenlerinin, durum makinesinin, veri akışının ve fail-safe mekanizmalarının tasarımda öngörüldüğü şekilde çalıştığını doğrulamayı amaçlamaktadır. Bu yaklaşım, sistemin simülasyondan gerçek ortama geçiş sürecinde güvenilir ve öngörülebilir davranış sergilemesini desteklemektedir.

4.0 Sonuç

Bu bölümde, SAÜRO İHA yazılım tasarım dokümanında ele alınan mimari yaklaşımlar, tasarım kararları ve güvenlik prensipleri genel bir değerlendirme çerçevesinde özetlenmektedir. Doküman boyunca tanımlanan yapı, 2025–2026 TEKNOFEST Döner Kanat İHA yarışma gereksinimleri ile uyumlu olacak şekilde; aynı zamanda yazılım mühendisliği açısından profesyonel ve sürdürülebilir bir sistem tasarımını hedeflemiştir.

4.1 Tasarımın Hedeflerle Uyumlu

Hazırlanan yazılım tasarımı; görev yürütme, algılama, iletişim ve güvenlik gereksinimlerini kapsayacak şekilde yapılandırılmıştır. ROS 2 tabanlı modüler mimari sayesinde yazılım bileşenleri birbirinden ayrıştırılmış, görev yönetimi durum makinesi üzerinden deterministik ve izlenebilir hale getirilmiştir.

Fail-safe mekanizmaları ve güvenlik kararları, görev başarısından önce uçuş emniyetini önceliklendirecek biçimde tasarlanmış; kamera veri akışı ve yer istasyonu iletişimi ise görev güvenliğini etkilemeyecek şekilde kontrollü olarak ele alınmıştır. Bu yaklaşım, sistemin hem yarışma senaryolarında hem de gerçek saha koşullarında güvenli biçimde çalışmasını hedeflemektedir.

4.2 Tasarım Kararlarının Gerekçeleri

Bu doküman kapsamında alınan tasarım kararları, yalnızca yarışma gereksinimlerini karşılamak amacıyla değil; yazılım mühendisliği açısından sürdürülebilir, genişletilebilir ve güvenli bir sistem ortaya koymak hedefiyle belirlenmiştir. Bu bölümde, tasarım sürecinde öne çıkan temel kararların arkasındaki gerekçeler özetlenmektedir.

- **ROS 2 Tabanlı Mimari Tercihi**

ROS 2, modüler yapısı, mesaj tabanlı iletişim modeli ve simülasyon-gerçek sistem uyumluluğu nedeniyle tercih edilmiştir. Bu yapı, yazılım bileşenlerinin bağımsız geliştirilmesini ve test edilmesini mümkün kılarken, ilerleyen aşamalarda yeni görev ve algılama modüllerinin sisteme entegre edilmesini kolaylaştırmaktadır.

- **Durum Makinesi ile Görev Yönetimi**

Görev yürütme sürecinin durum makinesi üzerinden yönetilmesi, görev akışının deterministik ve izlenebilir olmasını sağlamaktadır. Bu yaklaşım, karmaşık görev senaryolarında kontrolsüz akışların önüne geçerken; hata ve anormal durumlarda güvenli geçişlerin açık şekilde tanımlanmasına imkân tanımaktadır.

- **Yardımcı Bilgisayar – Uçuş Bilgisayarı Ayrımı**

Yardımcı bilgisayar (Raspberry Pi) ile uçuş bilgisayarı (Pixhawk) arasındaki sorumluluk ayrımı, uçuş güvenliğini artırmak amacıyla yapılmıştır. Kritik uçuş kontrol fonksiyonlarının uçuş bilgisayarı üzerinde tutulması, algılama ve görev mantığının ise yardımcı bilgisayarda yürütülmesi; sistemin güvenli ve kararlı çalışmasını desteklemektedir.

- **Hibrit Yer İstasyonu İletişimi**

Yer istasyonu ile iletişimde hibrit bir yaklaşım benimsenmiştir. Uçuş telemetrisi doğrudan uçuş bilgisayarı üzerinden aktarılırken, kamera verisi ve görev durumları yardımcı bilgisayar üzerinden iletilmektedir. Bu tasarım, telemetri hattının güvenilirliğini korurken, görev izlenebilirliğini artırmaktadır.

- **Kamera Veri Akışı Politikası**

Kamera veri akışı için tanımlanan çok seviyeli streaming modları, görev güvenliği ve sistem performansı gözetilerek belirlenmiştir. Varsayılan olarak kamera akışının kapalı olması, görev sırasında performans kaybını önlemekte; test ve doğrulama süreçlerinde ise kontrollü biçimde görsel veri elde edilmesine olanak sağlamaktadır.

- **Fail-Safe ve Güvenlik Yaklaşımı**

Fail-safe tasarımı, görev başarısından önce uçuş emniyetini önceliklendirecek şekilde ele alınmıştır. Fail-safe durumunun terminal bir durum olarak tanımlanması ve RTL + LAND davranışı ile sonuçlanması, sistemin anormal koşullarda öngörülebilir ve güvenli biçimde davranmasını sağlamaktadır.

4.2.1 Tasarımın Güçlü Yönleri

Hazırlanan tasarımın öne çıkan güçlü yönleri aşağıda özetlenmiştir:

- Görevden bağımsız, genişletilebilir durum makinesi yapısı
- ROS 2 node mimarisi ile sağlanan modülerlik ve test edilebilirlik
- Yardımcı bilgisayar ve uçuş bilgisayarı ayrımı sayesinde güvenli sistem mimarisi
- Kamera veri akışı için tanımlanan performans farkındalıklı streaming politikası
- Simülasyon ortamından gerçek sisteme geçişi destekleyen arayüz tabanlı tasarım yaklaşımı

Bu unsurlar, yazılımın yalnızca mevcut görevleri değil, ilerleyen aşamalarda eklenecek yeni görev ve algoritmaları da destekleyecek şekilde yapılandırıldığını göstermektedir.

4.3 Kısıtlar ve Bilinçli Sınırlar

Bu dokümanda bazı konular bilinçli olarak kapsam dışında bırakılmıştır. Algoritmik detaylar, görüntü işleme parametreleri, PID ayarları ve donanıma özgü tuning süreçleri bu tasarım dokümanının kapsamına dahil edilmemiştir.

Bu yaklaşım, yazılım tasarımının yüksek seviyeli mimari kararlar ve sistem davranışları üzerine odaklanmasını sağlamakta; uygulama ve optimizasyon detaylarının geliştirme ve test aşamalarında ele alınmasına imkân tanımaktadır.

4.4 Sonraki Aşamalar

Tasarım dokümanında tanımlanan yapı, uygulama ve test süreçleri için bir referans niteliği taşımaktadır. Sonraki aşamalarda:

- Konfigürasyon parametrelerinin saha testleri ile doğrulanması,
- Simülasyon senaryolarının genişletilmesi,
- Fail-safe eşiklerinin uçuş testleri ile rafine edilmesi,
- Görev çeşitliliğine bağlı olarak durum makinesinin genişletilmesi

planlanmaktadır. Bu adımlar, sistemin güvenilirliğini ve operasyonel uygunluğunu artırmayı hedeflemektedir.